

Atelier 1 : diagnostic initial

ShopFlow · PostgreSQL 18.6 · résultats et interprétation

Contexte. PostgreSQL 18.6 (Docker postgres:18, WSL2), base et schéma shopflow, chargés par 01_schema.sql et 02_donnees.sql : 1 000 clients, 200 produits, 100 000 commandes, 300 000 lignes. work_mem = 4 MB, shared_buffers = 128 MB. Aucun index ajouté : seuls ceux des contraintes (clés primaires, UNIQUE). Mesures à chaud, le 05/10/2026.

1. Les deux requêtes

Q1 : historique du client 42 (page d'API, très sélective)

```
SELECT id, created_at, total
FROM shopflow.commandes
WHERE client_id = 42
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

Q2 : indicateurs mensuels (lit toute la table)

```
SELECT date_trunc('month', created_at) AS mois,
       statut,
       count(*) AS nb_commandes,
       sum(total) AS chiffre_affaires
FROM shopflow.commandes
GROUP BY mois, statut
ORDER BY mois, statut;
```

2. Résultats fonctionnels vérifiés

On vérifie l'exactitude avant de mesurer : ces chiffres sont la référence qui doit rester identique, au centime, après toute optimisation (slides 6 et 24).

Req.	Lignes	Total	Contrôle de référence	OK
Q1	20	14 400,00 €	Le client 42 a 100 commandes (47 760,00 €), la page en garde 20	✓
Q2	30 groupes	29 775 001,25 €	sum(total) sur toute la table : 100 000 commandes, 29 775 001,25 €. Écart 0	✓

3. Mesures : 5 répétitions après échauffement

3 exécutions d'échauffement, puis 5 avec EXPLAIN (ANALYZE, BUFFERS). Execution Time en ms. Les 5 plans de chaque requête sont identiques.

Req.	#1	#2	#3	#4	#5	Médiane	Buffers
Q1	0,207	0,267	0,234	0,208	0,229	0,229	hit=99
Q2	84,3	84,1	80,5	86,2	78,7	84,1	hit=1 680, temp 380/381

Froid ou chaud

Première exécution de Q2, à froid : 1 096,8 ms (shared read=1330). À chaud : ≈ 84 ms (shared read=0). Même plan, 13 fois plus rapide : l'état du cache doit toujours être précisé (slide 11).

4. Nœud coûteux et hypothèse

Q2 : agrégation mensuelle

Nœud (lecture de bas en haut)	Temps propre	Part
Seq Scan on commandes (100 000 lignes)	≈ 25 ms	32 %
Sort : external merge, Disk 3 040 kB	≈ 37 ms	47 %
GroupAggregate (30 groupes)	≈ 15 ms	20 %

Nœud coûteux : le Sort. Il trie 100 000 lignes pour produire 30 groupes et déborde sur disque (381 pages temporaires). Le Seq Scan est normal, puisque toute la table est nécessaire.

Hypothèse : work_mem (4 MB) est trop petit pour ce tri, **et** le planificateur estime 80 048 groupes au lieu de 30, faute de statistique sur date_trunc('month', created_at). Il choisit donc de trier plutôt que de faire un HashAggregate en mémoire. **À tester ensuite :** SET work_mem = '32MB', puis une statistique sur l'expression.

Q1 : historique du client 42

Déjà rapide (≈ 0,23 ms) et l'estimation est exacte (100 lignes prévues, 100 lues). L'accès passe par l'index unique (client_id, cle_idempotence) issu d'une contrainte. **Nœud le plus coûteux : le Bitmap Heap Scan**, qui lit 88 pages pour 100 commandes dispersées, puis un tri n'en garde que 20. **Hypothèse :** le coût croît avec le nombre de commandes du client, parce que l'index ne contient pas created_at. Piste pour la journée 2 : un index sur (client_id, created_at DESC, id DESC).

Annexe : plans EXPLAIN (ANALYZE, BUFFERS) complets (mesure n°5)

Q1 : historique du client 42

```
Limit (cost=329.84..329.89 rows=20 width=22) (actual time=0.183..0.186 rows=20.00 loops=1)
  Buffers: shared hit=99
  -> Sort (cost=329.84..330.09 rows=100 width=22) (actual time=0.182..0.184 rows=20.00 loops=1)
    Sort Key: created_at DESC, id DESC
    Sort Method: top-N heapsort Memory: 27kB
    Buffers: shared hit=99
    -> Bitmap Heap Scan on commandes (cost=5.07..327.18 rows=100 width=22) (actual time=0.039..0.147 rows=100.00 loops=1)
      Recheck Cond: (client_id = 42)
      Heap Blocks: exact=88
      Buffers: shared hit=93
      -> Bitmap Index Scan on commandes_client_id_cle_idempotence_key (cost=0.00..5.04 rows=100 width=0) (actual time=0.000..0.000 rows=100.00 loops=1)
        Index Cond: (client_id = 42)
        Index Searches: 1
        Buffers: shared hit=5
Planning:
  Buffers: shared hit=125
Planning Time: 0.485 ms
Execution Time: 0.229 ms
```

Q2 : agrégation mensuelle

```
GroupAggregate (cost=13279.82..15730.54 rows=80048 width=54) (actual time=54.107..77.900 rows=30.00 loops=1)
  Group Key: (date_trunc('month'::text, created_at)), statut
  Buffers: shared hit=1680, temp read=380 written=381
  -> Sort (cost=13279.82..13529.82 rows=100000 width=20) (actual time=53.828..62.580 rows=100000.00 loops=1)
    Sort Key: (date_trunc('month'::text, created_at)), statut
    Sort Method: external merge Disk: 3040kB
    Buffers: shared hit=1680, temp read=380 written=381
    -> Seq Scan on commandes (cost=0.00..2924.00 rows=100000 width=20) (actual time=0.760..25.395 rows=100000.00 loops=1)
      Buffers: shared hit=1674
Planning:
  Buffers: shared hit=152
Planning Time: 0.490 ms
Execution Time: 78.664 ms
```